

Three possible BiLSTM-based approaches for Event Detection

MNLP - Homework 1 Report

Antonello Giorgio

1836529

giorgio.1836529@studenti.uniroma1.it

1 Introduction

Event detection is a NLP task that requires the analysis of all the input text, rather than just taking into consideration words separately for the computation of the result, which is an event-label assigned to each word: therefore, it's very crucial to **keep information of the context** while performing the prediction model.

Given this assumption, I decided to implement a model based on the **Bidirectional Long-Short Term Memory (Bi-LSTM)** architecture. Moreover, I tried to perform the algorithm with the addition of:

1. *word embeddings*;
2. *word embeddings* concatenated with *Part-Of-Speech (POS) embeddings*, passed as input to the Bi-LSTM;
3. *word embeddings* concatenated with *Part-Of-Speech (POS) embeddings*, passed as input to the Bi-LSTM, and *Conditional Random Fields (CRF)*, applied on the output of the Bi-LSTM.

2 Methodology

2.1 Analysis and Preprocessing of the Dataset

Given a sentence, which is a list of tokens (words as strings), the task essentially consists in assigning a label, belonging to the event set (SEN:Sentiment, CHA:Change, ACT:Action, SCEN:Scenario, POSS:Possession), to each token of the sentence. Each dataset split adopts the **BIO format** to represent event trigger spans, so we actually have **11 possible labels** (see Table 1).

From the plot of the distribution of the event labels among the training sentences (see Figure 1), we can see the **predominance of the 'O' class**, with a less pronounced presence, but certainly higher w.r.t. the others, of the **"B-ACTION" and**

"B-CHANGE" classes: this lets us suppose that the model should perform better on these classes, and the tokens labeled with the other event classes will be much more difficult to correctly classify (they will most likely be classified as "O", since the training set is unbalanced precisely towards that class).

Before passing the data directly to the model, I created three **vocabularies** (from token to index): one from the training data, one for the event labels and the last one for the POS tags. Moreover, I **padded the sentences**, in order to pass same-size sequences to the model. To make the vocabulary invariant w.r.t. words with capital letters, I've built it with the tokens from the training sentences **converted into lowercase format**: even though I've thought it could be a good improvement, the results showed that it didn't change much the results, it rather made them worse w.r.t. the results obtained by the model with same hyperparameters and a vocabulary not modified in that sense (see subsection 3.2.1 and subsection 3.2.2 from Experiments section with BiLSTM+POS).

2.2 Word Embeddings

It is commonly known the use of word embeddings to better represent the common features of different words in a multidimensional vectorial space, in such a way that **related words are nearer as vectors** in the feature space. To retrieve them, I've imported the *gensim models* (Řehůřek, 2022), which contains lots of different word embeddings with different dimensions. Because of hardware limitations, I couldn't use *Fasttext*, *Conceptnet*, *Word2Vec ruscorpora 300* and *Word2Vec 300* embeddings (too much RAM needed); instead, I've trained my model with **GloVe weights** and I've found out that the ones with smaller dimensions make the model perform better (under the same conditions and hyperparameters, see Figure 2, 3, 4 and 5 for comparisons). For the final experiments

I used only GloVe-50 embeddings, since they are the ones that give the best results: this could be due to a **more complexity of the model when we introduce embeddings of higher dimensions**, that significantly decrease the performances for this kind of task and introduce overfitting.

2.3 POS Embeddings

POS embeddings identify a word's **syntactic function** in a phrase, such as whether it is a noun, verb, adjective, or adverb. The **grammatical structure** of the text, which is frequently **crucial for recognizing events**, can be better captured by the model by using this data, hence I created an alternative version of the model, in which the Bi-LSTM takes a concatenation of the *NLTK* (Steven Bird and Loper, 2019) POS embeddings with GloVe-50 word embeddings as input.

2.4 Bi-LSTM

The **Bi-LSTM layer** takes the word embeddings of the sample sentence (or the concatenated embeddings, if it's the implementation with also POS embeddings) as input and processes them sequentially, updating its hidden state at each time step. By processing the input sequence in both directions, The Bi-LSTM is able to capture **long-term dependencies** and **contextual information from both sides of the input sequence**: this enables the network to capture both the **past and future contexts** of each word, which can be helpful for more effectively detecting events.

The output of the Bi-LSTM is passed to a linear fully-connected **classifier layer** that produces a prediction for each word, indicating whether it is part of an event or not (either it belongs to one of the five event categories in the dataset, or it's 'O').

To prevent overfitting and increase the generalization power of the model, I introduced a **dropout layer** that randomly drops out both some of the input embeddings to the Bi-LSTM layer and some of the outputs of the Bi-LSTM to the classifier layer.

2.5 Conditional Random Fields (CRF)

Another variant of the model consists in passing the output of the Bi-LSTM layer through a linear layer to generate a sequence of *emissions*, representing the likelihood of each word being part of an event. These emissions are then passed through a **Conditional Random Field (CRF) layer** (implemented with *pytorch-crf* module, see (Kurniawan, 2019)): it computes the *negative log-likelihood* loss

between the predicted emissions and the true labels, and returns the final predictions for the input sequence. Each prediction is influenced by both the probability of each word and the probabilities of its neighboring words: in fact the CRF layer **models the dependencies between adjacent words** in the sequence, allowing it to take into account the context and dependencies between words when making predictions. That's the reason why I opted for including them.

3 Experiments

Here there are some of the best experiments with three types of model implementations. To compute the F1-Score over the valid/test data, I used the Python package *segeval* (Nakayama, 2018).

3.1 Bi-LSTM with only Word Embeddings

See Table 2

3.2 Bi-LSTM with Word+POS Embeddings

3.2.1 Experiment 1

See Table 3

3.2.2 Experiment 2

See Table 4

3.3 Bi-LSTM with Word+POS Embeddings and CRF

3.3.1 Experiment 1

See Table 5

3.3.2 Experiment 2

See Table 6

4 Conclusion

The **results** with these three possible implementations of the prediction model, applied on the given train, dev and test data, showed that, even with no significant changes in terms of F1-Score metric, the **best performance is associated with medium model complexity**: as expected, the introduction of word embeddings concatenated with POS embeddings improved the performances, while CRFs didn't unexpectedly help detecting event-related words better, even if their purpose is strictly related to the further consideration of the neighbors of each word to give a prediction.

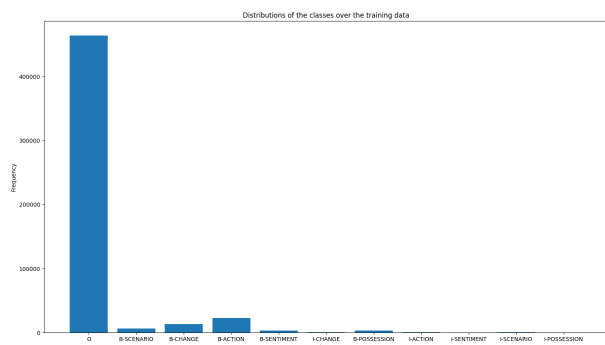


Figure 1: Distribution of the event labels over the training set.

References

Kemal Kurniawan. 2019. [Conditional random fields with pytorch](#).

Hiroki Nakayama. 2018. [sequeval: A python framework for sequence labeling evaluation](#). Software available from <https://github.com/chakki-works/sequeval>.

Ewan Klein Steven Bird and Edward Loper. 2019. [Nltk pos tags](#).

Radim Řehůřek. 2022. [Gensim embeddings](#).

Table 2: Bi-LSTM with only Word Embeddings

Hyperparameters	Table 7
Loss/F1 Plot	Figure 6
Confusion Matrix	Figure 7
Test F1-Score	0.7040

Table 3: Bi-LSTM with Word+POS Embeddings - Experiment 1

Hyperparameters	Table 8
Loss/F1 Plot	Figure 8
Confusion Matrix	Figure 9
Test F1-Score	0.7072

Table 4: Bi-LSTM with Word+POS Embeddings - Experiment 2

Hyperparameters	Table 9
Loss/F1 Plot	Figure 10
Confusion Matrix	Figure 11
Test F1-Score	0.7085

Table 5: Bi-LSTM with Word+POS Embeddings and CRF - Experiment 1

Hyperparameters	Table 10
Loss/F1 Plot	Figure 12
Confusion Matrix	Figure 13
Test F1-Score	0.7046

Table 6: Bi-LSTM with Word+POS Embeddings and CRF - Experiment 2

Hyperparameters	Table 11
Loss/F1 Plot	Figure 14
Confusion Matrix	Figure 15
Test F1-Score	0.7045

Table 7: Hyperparameters table for the simple BiLSTM model experiment

Hyperparameters	Values
Epochs	50
Batch size	32
Learning rate	0.001
Hidden dimension	128
Number of BiLSTM layers	5
Dropout	0.3
Lowercase tokens	False

Table 1: The 11 possible event labels

B-SENTIMENT	I-SENTIMENT
B-CHANGE	I-CHANGE
B-ACTION	I-ACTION
B-SCENARIO	I-SCENARIO
B-POSSESSION	I-POSSESSION
O	

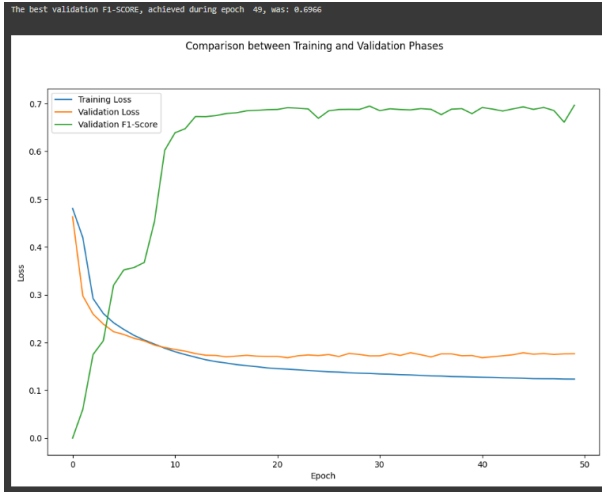


Figure 2: Train/Valid Loss and Valid F1-Score plots, with basic Bi-LSTM model and GloVe 50 word embeddings. The behavior of the Valid Loss over the epochs is promising, with a F1-Score that has a sudden peak after few epochs and a steady behavior after.

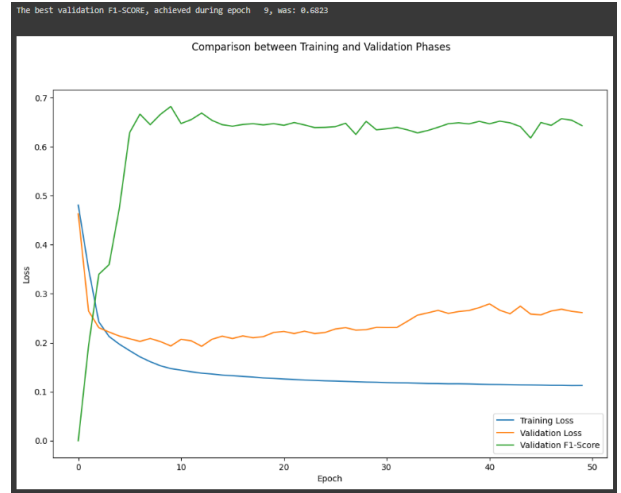


Figure 4: Train/Valid Loss and Valid F1-Score plots, with basic Bi-LSTM model and GloVe 200 word embeddings. From the train/valid losses behaviors, we can observe a principle of overfitting, with a F1-Score that also starts to decrease.

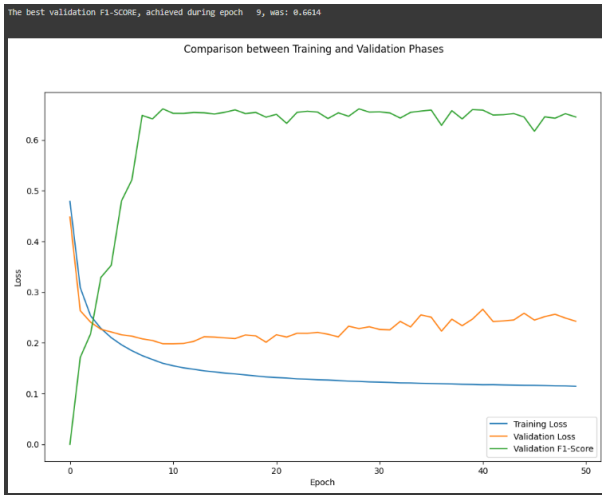


Figure 3: Train/Valid Loss and Valid F1-Score plots, with basic Bi-LSTM model and GloVe 100 word embeddings. The Valid Loss starts to increase over the epochs, with no significant change of F1-Score behavior w.r.t. GloVe 50.

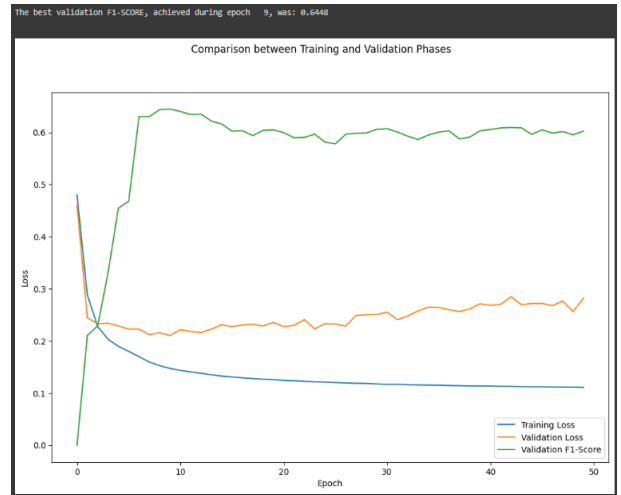


Figure 5: Train/Valid Loss and Valid F1-Score plots, with basic Bi-LSTM model and GloVe 300 word embeddings. Here the overfitting is very clear, with the F1-Score that remains constant after the peak and the decrease (also seen in GloVe 200).

Table 8: Hyperparameters table for the 1st experiment with BiLSTM+POS model

Hyperparameters	Values
Epochs	100
Batch size	128
Learning rate	0.001
Hidden dimension	128
Number of BiLSTM layers	3
Dropout	0.5
Lowercase tokens	True
POS embeddings dimension	50

Table 9: Hyperparameters table for the 2nd experiment with BiLSTM+POS model

Hyperparameters	Values
Epochs	100
Batch size	128
Learning rate	0.001
Hidden dimension	128
Number of BiLSTM layers	3
Dropout	0.5
Lowercase tokens	False
POS embeddings dimension	50

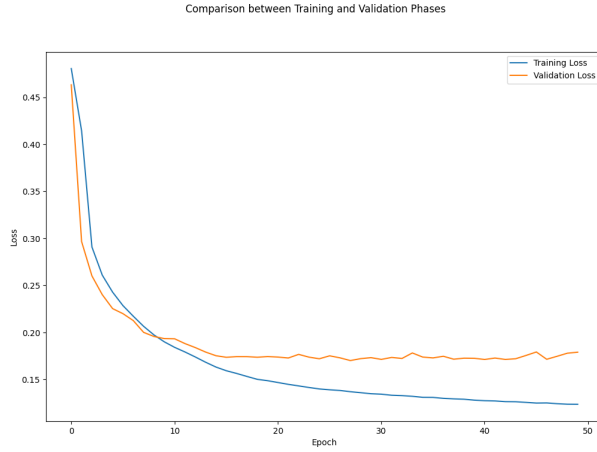


Figure 6: Plot of the train and valid losses for the experiment with the Bi-LSTM and only word embeddings. Trying with epochs higher than 50, it would result in an increasing validation loss, so for this kind of simpler model it's better to stop the training here.

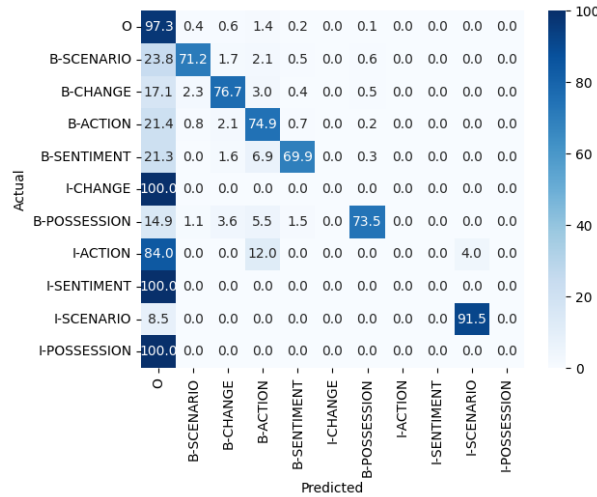


Figure 7: Confusion matrix of the Bi-LSTM model with only word embeddings. As expected, 'O' has the highest score, 'B-ACTION' and 'B-CHANGE' have higher scores w.r.t. the rest of the classes, especially with the I- classes, which have too few samples seen in training phase; unexpectedly, 'I-SCENARIO' class the second highest score, even with very few samples seen in training phase.

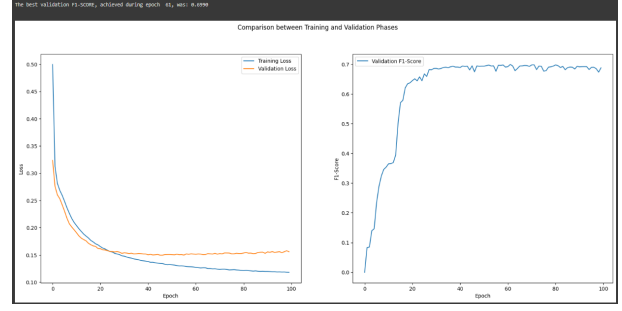


Figure 8: Plot of the train and valid losses and valid F1-Score for the 1st experiment with the Bi-LSTM+POS. Even though there's the adding of lowercase tokens in vocabulary, there's no improvement w.r.t. the 2nd experiment, which has the same hyperparameters but no lowercase. The F1-Score is even a bit worse.

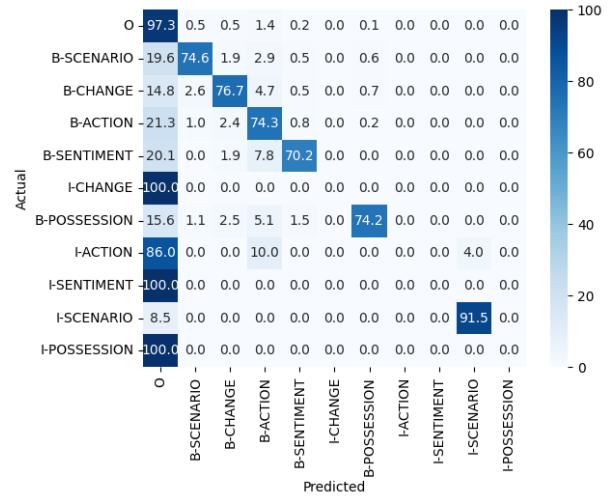


Figure 9: Confusion matrix of the 1st experiment with Bi-LSTM+POS model. Similar behavior as the one of the simple Bi-LSTM model. Improvements in 'B-SCENARIO' class.

Table 10: Hyperparameters table for the 1st experiment with BiLSTM+POS+CRF model

Hyperparameters	Values
Epochs	100
Batch size	128
Learning rate	0.001
Hidden dimension	128
Number of BiLSTM layers	3
Dropout	0.5
Lowercase tokens	False
POS embeddings dimension	50

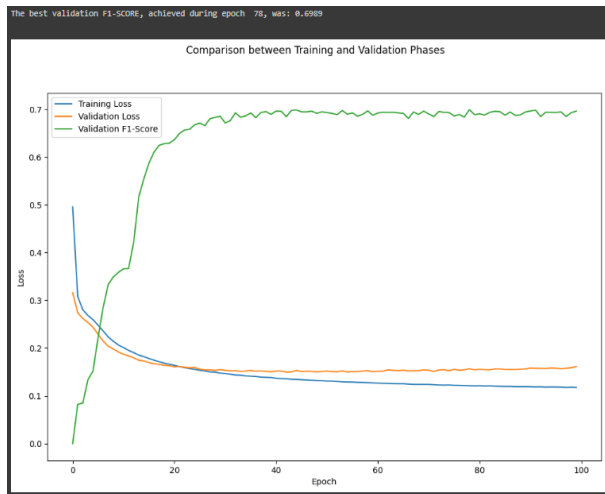


Figure 10: Plot of the train and valid losses and valid F1-Score for the 2nd experiment with the Bi-LSTM+POS. It is the best behavior of all the models, both of losses and F1-Score, which reaches the peak of 0.7085.

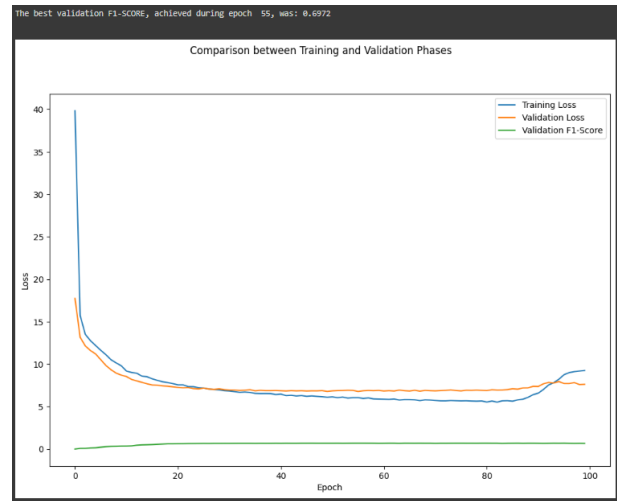


Figure 12: Plot of the train and valid losses and valid F1-Score for the 1st experiment with the Bi-LSTM+POS+CRF. Although the model has the highest scores in terms of classes accuracies, the behavior of the loss is bad after epoch 80, because the training loss starts to increase, maybe due to the fact that the model is trying to adapt to the train data too much => overfitting!

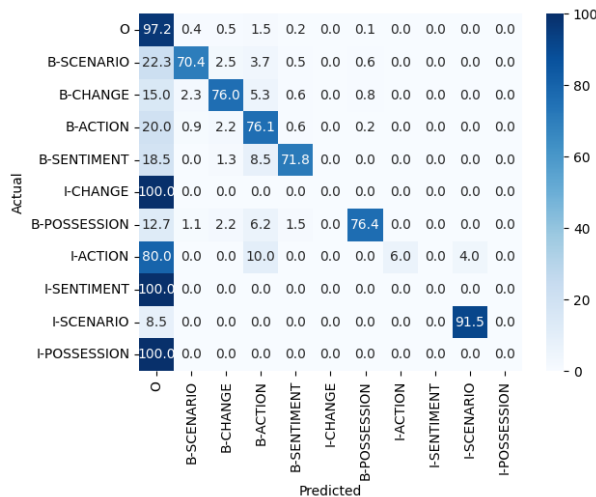


Figure 11: Confusion matrix of the 2nd experiment with Bi-LSTM+POS model. As one would have expected, it should have the highest scores among all the different models; instead, it is second only to the 1st experiment of BiLSTM+POS+CRF. Apart from this, it has improvements in some classes, in which now it is at least predicted labels different from 'O': a good sign that words are at least recognized as part of events.

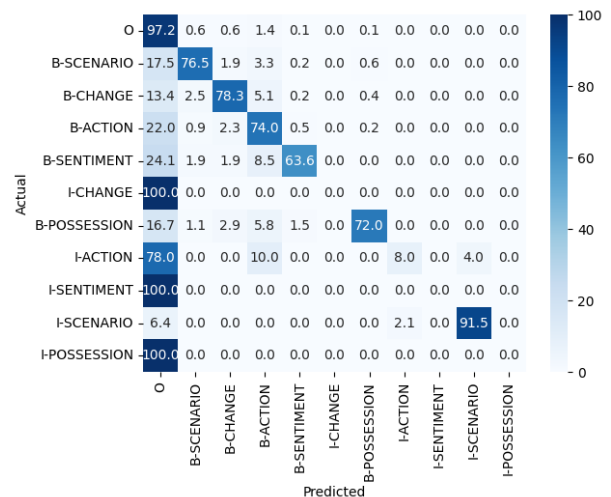


Figure 13: Confusion matrix of the 1st experiment with Bi-LSTM+POS+CRF model. It has the highest scores among all the different models, even though it still finds difficult to predict correctly I-classes (except from "I-SCENARIO" and "I-ACTION", which has an improvement here.)

Table 11: Hyperparameters table for the 2nd experiment with BiLSTM+POS+CRF model

Hyperparameters	Values
Epochs	100
Batch size	32
Learning rate	0.01
Hidden dimension	128
Number of BiLSTM layers	3
Dropout	0.4
Lowercase tokens	False
POS embeddings dimension	50

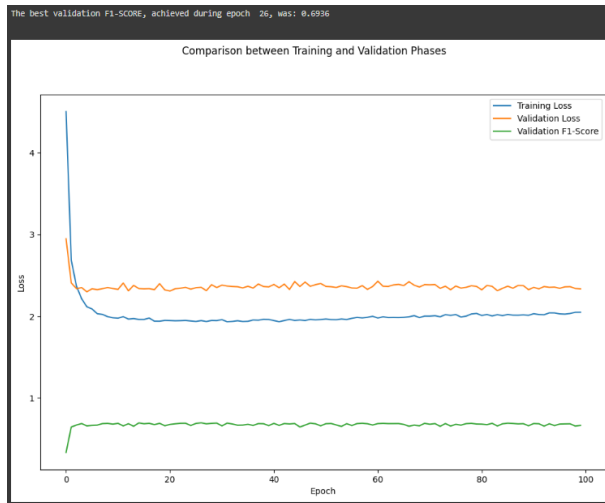


Figure 14: Plot of the train and valid losses and valid F1-Score for the 2nd experiment with the Bi-LSTM+POS+CRF. Here the losses are very high from the beginning, then they start to stabilize themselves on values between 2 and 3. The F1-Score reaches the peak very quickly, then it starts to slowly and constantly decrease. One of the worst behaviors of the experiments seen here.

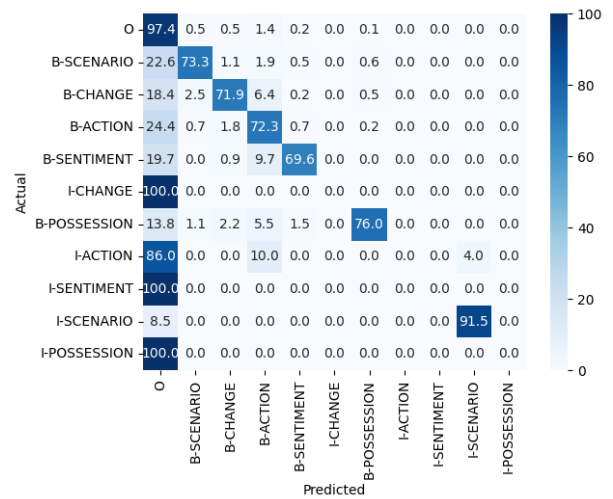


Figure 15: Confusion matrix of the 2nd experiment with Bi-LSTM+POS+CRF model. Here it comes back to the values of the first simpler model: in fact, in terms of accuracy, it has very lower values w.r.t. the first experiment of the same model (e.g. 'I-ACTION' goes from 1st experiment = 8.0 to 2nd experiment = 0.0).